ICT ACADEMY OF KERALA

PROJECT REPORT

LifeLedger 2.0

An Intelligent Product Lifecycle, Warranty Tracking,
and Ownership Cost Management System

SUBMITTED BY

Joyel Joshy

Sahrdaya College of Engineering and Technology

Academic Year 2026–27

Full-Stack Web Application Development

EXECUTIVE SUMMARY

LifeLedger 2.0 is a proposed web-based product lifecycle and warranty management system intended to help users keep product information, warranty details, purchase receipts, repair history, and ownership expenses in one place. Instead of searching through separate documents and messages, a user would be able to refer to a centralized product record and review important ownership information.

The intended technology stack is MERN: MongoDB for persistent data storage, Express.js for backend API development, React.js for the user interface, and Node.js for the server runtime. Depending on the current source code, supporting technologies may include Vite, Mongoose, JWT, bcrypt, Multer, Git, and GitHub. The project repository is identified as the Life-Ledger repository on GitHub.

This report documents the project concept, objectives, intended modules, proposed architecture, development approach, and verification needs. The source code and live deployment have not been provided for inspection as part of this report preparation; therefore, the document does not claim that every described module is already implemented or that deployment has been verified.

1. INTRODUCTION

People often manage product details using paper receipts, email messages, warranty cards, and separate notes. Over time, it can become difficult to find the purchase date, confirm whether a warranty is still active, or remember how much has been spent on repairs. LifeLedger 2.0 is intended to address this problem by organizing product ownership information through a single web application.

The system focuses on product records and the costs associated with owning products. For example, if a laptop costs ■50,000 and later requires ■2,000 in repairs, the total recorded ownership cost would be ■52,000. The system aims to make these details easier to access and understand.

2. PROJECT OBJECTIVES

- Create a centralized place to organize purchased-product information.
- Provide a dashboard that summarizes relevant product records and ownership details.
- Support adding, viewing, updating, and—if implemented—deleting product records.
- Record warranty dates and help users identify active and expired warranties.
- Organize purchase receipts associated with individual products, subject to the implemented upload and storage design.
- Maintain repair and service records, including dates, descriptions, and costs, if supported by the implementation.
- Calculate or summarize ownership cost using the purchase price and available additional expenses.
- Use a maintainable full-stack architecture and keep sensitive configuration outside committed source code.

3. SCOPE AND DELIVERABLES

The intended scope of LifeLedger 2.0 covers product lifecycle information from purchase through warranty monitoring and service or repair tracking. A product record may include product name, brand, model, purchase date, purchase price, and warranty information. Additional records may include receipts and service costs. The exact fields, validation rules, and operations must be confirmed against the current application code.

Expected project deliverables include a web-based user interface, backend API, database models, product-management workflow, warranty information handling, ownership-cost calculation, documentation, and a source-code repository. The project should be described as a product lifecycle and ownership-cost management application; future enhancements should not be presented as completed functionality unless they have been implemented and tested.

The following are proposed future enhancements unless verified in the source code: OCR-based receipt scanning, automatic warranty-expiry reminders, repair-versus-replacement comparison, digital product passports, e-waste recycling guidance, and advanced cost analytics.

4. TECHNOLOGY STACK

Technology	Intended purpose
React.js	Builds the browser-based user interface and interactive product views and forms.
Node.js	Runs the server-side JavaScript application.
Express.js	Provides backend routing and API endpoints for application operations.
MongoDB	Stores product, user, warranty, and service information, according to the final data model.
Mongoose (possible)	May provide schemas and database operations between the Node.js backend and MongoDB.
Vite (possible)	May provide the React development server and frontend build tooling.
JWT / bcrypt (possible)	May support authentication tokens and password hashing if implemented.
Multer (possible)	May support receipt or document uploads if file upload is implemented.
Git and GitHub	Track source-code changes and host the project repository.

Note: The stack above describes the intended design. Supporting packages and actual usage must be verified in the repository before being reported as implemented.

5. SYSTEM ARCHITECTURE

The intended architecture separates the user interface, backend application logic, and persistent data storage. The frontend collects input and displays results; the backend validates and processes requests; and the database stores and retrieves records.

FRONTEND	BACKEND	DATABASE
React.js Product forms and dashboard HTTP API requests	Node.js + Express.js Request handling and business logic Validation and API responses	MongoDB Product and related records Persistent storage

Conceptual data flow: User → React Frontend → Express API running on Node.js → MongoDB Database.

6. MAIN SYSTEM MODULES

6.1 Dashboard

The dashboard is intended to give users a quick overview of their saved products and relevant information. The precise cards, counts, and summaries depend on the features implemented in the current frontend.

6.2 Product Management

This module is intended to let users add and view product records and update existing details. Deletion may also be provided. Possible fields include product name, brand, model, purchase date, and purchase price; the actual database schema and form fields must be checked in the source code.

6.3 Warranty Tracking

Warranty tracking is intended to store warranty-related dates and help users determine whether coverage is active or expired. Reliable status calculation depends on the warranty fields and date-handling logic implemented in the application.

6.4 Receipt Management

The planned receipt module organizes purchase receipts with the related product. Actual upload support, accepted file types, file storage location, and access controls must be verified before describing receipt uploads as a working feature.

6.5 Repair and Service History

This module is intended to maintain repair or maintenance entries, such as the service date, description, and cost. The available operations and stored fields depend on the implemented data model.

6.6 Ownership Cost Management

Ownership cost can be calculated by adding the purchase price to recorded repair and service expenses. For example: purchase price ■50,000 + repair expense ■2,000 = total recorded ownership cost ■52,000. If other expense categories are added later, the formula can be expanded to include them.

7. DEVELOPMENT METHODOLOGY

The project can be developed and improved in stages. First, confirm the existing repository structure and requirements. Next, review the React interface and backend routes, check the database models, and connect the frontend to the correct API endpoints. Then verify the main product workflow, warranty-date logic, receipt handling, and cost calculations. Finally, document the observed results and test the deployment configuration.

- Requirements review: list the workflows and identify which ones already exist.
- Frontend review: inspect pages, forms, navigation, and validation.
- Backend review: inspect API routes, request handling, and error responses.
- Database review: confirm schemas, relationships, and stored fields.
- Integration and verification: test complete user actions from the interface to the database.
- Version control: use Git and GitHub while keeping credentials out of the repository.

Because the source code has not been inspected for this report, this methodology is presented as a development and verification approach rather than a claim that every step has already been completed.

8. REPOSITORY AND DEPLOYMENT

The project repository provided in the project brief is:

GitHub repository: https://github.com/joyel225749-ship-it/Life-Ledger

Intended GitHub Pages website: https://joyel225749-ship-it.github.io/Life-Ledger/

The deployment plan described in the project context considers GitHub Pages for the frontend, Render for the backend, and MongoDB Atlas for the database. The current deployment status has not been confirmed. GitHub Pages can host a static frontend, but full application functionality requires the backend API and database to be correctly configured and reachable.

Before presenting a live deployment as complete, check the GitHub Actions workflow and repository settings, open the public frontend, confirm the backend service is running, verify environment variables in the hosting service, and test that frontend requests reach the backend. Never publish the real .env file, database password, JWT secret, or API keys.

9. RESULTS AND VERIFICATION

The supplied project context describes the intended purpose, technology stack, modules, and deployment plan, but it does not include the application source code or verified test evidence. For that reason, this report does not claim that the dashboard, CRUD operations, warranty calculations, file uploads, authentication, or deployment have been confirmed to work.

The following checks should be completed against the actual application:

- Create a product with valid information and confirm it is saved.
- Open and edit a product, then confirm the updated values persist.
- Check product deletion, including any confirmation step and error handling.
- Test warranty dates that are active, expired, and missing or invalid.
- If receipt upload exists, test allowed file types, file size limits, storage, and retrieval.
- Add repair/service expenses and confirm the ownership-cost total is correct.
- Test empty fields, invalid values, server errors, and network failures.
- Verify the deployed frontend can communicate with the deployed backend and database.

Only results actually observed during testing should be added to the final submitted report. Screenshots, test-pass counts, API paths, and database fields should not be invented.

10. SECURITY AND DATA HANDLING

The application may store personal account information, product details, receipts, and expenses. If authentication is implemented, passwords should be securely hashed and access to user-specific records should be checked on the backend. Uploaded receipts should be validated and stored safely. Secrets and database connection strings should be supplied through environment variables and excluded from source control. These are recommended practices; the current implementation must be inspected to confirm which protections are in place.

11. LIMITATIONS AND FUTURE ENHANCEMENTS

The current status of the code has not been independently verified for this report. Therefore, the main limitation of this document is that it describes the intended LifeLedger 2.0 design rather than a confirmed feature-by-feature implementation audit. Actual routes, schemas, authentication, upload behavior, tests, and deployment should be documented only after checking the repository and running the application.

Potential future enhancements include:

- OCR-based extraction of information from uploaded receipts.
- Automatic notifications before warranty expiry.
- Repair-versus-replacement cost comparison.
- Digital product passports containing product lifecycle information.
- E-waste recycling guidance when a product reaches end of life.
- Cost reports, charts, and analytics across product categories.
- Improved accessibility, input validation, error handling, and automated testing.

12. CONCLUSION

LifeLedger 2.0 aims to provide a centralized way to manage product information, warranty periods, receipts, service history, and ownership expenses. Its intended MERN architecture separates the React frontend, Express and Node.js backend, and MongoDB database. The ownership-cost example demonstrates how purchase and repair expenses can be combined into a useful total.

The next step is to inspect the actual project files, verify the modules already implemented, and test the complete data flow before finalizing implementation claims. With reliable validation, secure configuration, testing, and deployment verification, the project can be developed into a useful product ownership management application.

APPENDIX A. PROJECT LINKS

Application repository: https://github.com/joyel225749-ship-it/Life-Ledger

Intended frontend website: https://joyel225749-ship-it.github.io/Life-Ledger/

APPENDIX B. TECHNICAL REFERENCES

- React documentation: https://react.dev/
- Node.js documentation: https://nodejs.org/docs/
- Express documentation: https://expressjs.com/
- MongoDB documentation: https://www.mongodb.com/docs/
- Mongoose documentation: https://mongoosejs.com/docs/
- Vite guide: https://vite.dev/guide/

APPENDIX C. LOCAL SETUP NOTE

The exact setup commands should be taken from the package.json scripts and the current repository structure. In general, install Node.js and Git, obtain the repository, install dependencies in the appropriate frontend and backend folders, configure required environment variables locally, and start the services using the scripts defined by the project. Do not publish secrets or credentials.

Prepared by: Joyel Joshy | Sahrdaya College of Engineering and Technology | Academic Year 2026–27